

Databases II

Semester 4 (Theory)

Mathis Neunzig

DHBW Mosbach

Autumn 2026

Table of Contents

- 1 Introduction
- 2 Business Intelligence & Data Warehouse
- 3 DWH Architectures
- 4 Data Modelling
- 5 Building a Data Warehouse
- 6 OLAP & Business Intelligence

Table of Contents

- 1 Introduction
- 2 Business Intelligence & Data Warehouse
- 3 DWH Architectures
- 4 Data Modelling
- 5 Building a Data Warehouse
- 6 OLAP & Business Intelligence

About the Lecturer

Mathis Neunzig

- B.Sc. Business Information Technology (DHBW Mannheim)
- M.Sc. Data Science (Fernuniversität Hagen)
- ABAP Developer & Trainer at SAP
- Teaches Databases & Software Engineering at DHBW
- Based in Mannheim (Baden-Württemberg)
- Originally from Oldenburg (Lower Saxony)

Contact

Email

mathis.neunzig@gmail.com

Phone

+49 174 9885992

Getting to Know You

Let's go around the room – please tell us:

- 1 Your **name**
- 2 Your **employer**
- 3 Your **favourite animal**
- 4 Your **favourite song, album or artist** right now
- 5 Your **favourite programming language** (so far)

Agenda

This course is structured around multiple interconnected topic groups that build on each other progressively throughout the semester.

1. Foundations

- Databases and Database Management Systems (DBMS)
- Data models
- Relational algebra
- Relational databases
- Data modelling & the DB design process

2. SQL

- Data Manipulation Language (DML)
- Data Definition Language (DDL)
- Data Control Language (DCL)

3. Advanced Topics

- Database transaction management
- Concurrency control
- Performance optimisation
- Backup & Recovery
- Security & SQL Injection
- Object-Relational Mapping (ORM)

4. DB Project (4th semester)

- Recap of DB design process
- Recap of ORM
- Case study: designing a database for a real-world scenario

Semester Schedule

Session	Date	Time	Topics
1	09.09	12:30–15:00	Introduction; Course overview; Exam info; DB Management System; Data Models
2	15.09	14:00–17:15	Relational Databases; Basic Relational Algebra
3	19.09	09:00–12:15	Advanced Relational Algebra; Normal Forms (1NF, 2NF, 3NF)
4	26.09	09:00–12:15	Normal Forms (BCNF, 4NF, 5NF, 6NF)
5	29.09	14:00–17:15	ERM Design
6	13.10	14:00–17:15	SQL (DML)
7	20.10	14:00–17:15	SQL (DML)
8	27.10	14:00–17:15	SQL (DCL); CAP-Theorem
9	03.11	14:00–17:15	Backup & Recovery; Performance; Concurrency
10	10.11	14:00–17:15	Security; Object-Relational Mapping
11	17.11	14:00–17:15	Exam preparation
12	26.11	10:00–11:00	Written Exam

Mailing List & Contact

- All course announcements will be sent via discord.
- There is no mailing list in place for this class. Please check the discord channel regularly for updates.
- Lecturer contact: `mathis.neunzig@gmail.com` +49 174 9885992

Emergency Contact

If you have an urgent issue that cannot wait for the next session – for example a technical problem on the day of an assignment submission – please use the phone number above. For non-urgent matters, email or discord is always preferred.

General Conduct

Standard DHBW rules of conduct apply throughout all sessions.

- **Absences:** Please give advance notice if you cannot attend. A quick message the day before is always appreciated. If you miss a session, it is your responsibility to catch up on the material.
- **Deadlines:** If you foresee that you cannot meet an assignment deadline, contact the lecturer *before* the deadline, not after. Extensions can often be arranged if asked in advance.
- **Open door:** Feel free to reach out at any time with questions, concerns, or feedback; about the content, the pace, or anything else. There are no silly questions in this course.
- **Group exercises:** Active participation during group exercises is expected and contributes to your own learning as well as the group's. Sitting passively during a group task is not acceptable.

Assessment: Written Exam

Key Facts

- **Weight:** 40 points (40 % of final grade)
- **Timing:** End of 3rd semester (12, 26.11)
- **Duration:** 60 minutes

Exam Scope

Topics marked with (!) in the slides will *not* appear in the written exam. These are advanced or supplementary topics included for completeness and professional context, but you will not be tested on them under exam conditions.

Examples

Permitted Aids You may bring **one handwritten A4 sheet** (both sides allowed) as a personal cheat sheet. It must be handwritten – printed or typed sheets are not permitted. Writing your own cheat sheet is itself an effective revision strategy.

Assessment: Assignments and Project

Overview – 60 points total (60 % of final grade)

Assignment	Description	Points
<i>Semester 3</i>		
Assignment 1	Individual work	10 pts
Assignment 2	Individual work	10 pts
<i>Semester 4</i>		
Project	Final project (in groups)	40 pts

Submission Policy (Assignments 1 and 2)

- **1st submission** receives a grade *and* written feedback.
- **Optional 2nd submission** is possible after reviewing the feedback.
- The 2nd submission grade replaces the 1st – use this opportunity wisely.

No textbook purchase is required to pass this course.

All essential material is covered in the lecture slides and exercises.

Recommended reading (for those who want to go deeper):

- Kemper & Eickler – *Datenbanksysteme*
The German-language standard reference for database courses at German universities.
Thorough and mathematically precise.
- Garcia-Molina, Ullman & Widom – *Database Systems: The Complete Book*, 2nd ed.
Comprehensive English-language textbook covering theory and practice in depth.
- Foster & Godbole – *Database Systems: A Pragmatic Approach*, 2nd ed.
A more application-oriented introduction, well-suited as a companion to this course.
- Gumm & Sommer – *Einführung in die Informatik*, 10th ed.
Broader CS introduction, useful for contextualising databases within the wider field.

Table of Contents

- 1 Introduction
- 2 Business Intelligence & Data Warehouse
- 3 DWH Architectures
- 4 Data Modelling
- 5 Building a Data Warehouse
- 6 OLAP & Business Intelligence

What is Business Intelligence?

Definition

Business Intelligence (BI) refers to all tools and applications with a **decision-supporting character** that lead to better insight into one's own business and a deeper understanding of the mechanisms in relevant chains of causality. (Weber et al., 1999)

In plain terms

BI is the **intelligence service of the company**. It is the process that generates knowledge from fragmented, heterogeneous corporate, market, and competitive data about one's own and others' positions, potentials, and perspectives.

Two Ways to Think About BI

Wide BI Understanding (Technology & Application)

- ETL, Data Warehouse, OLAP, Reporting
- Standard & ad-hoc reporting, EIS/DSS/MIS systems
- Data Mining, Text Mining
- Planning, Consolidation, Balanced Scorecard
- Analytical CRM

Narrow BI Understanding (Analytics Focus)

- Focused on KPI systems and decision support
- Management reporting and scorecards
- Analytical applications for specific business domains

Picture: BI Facets Diagram

Add the “Facets of Business Intelligence” diagram showing the phases (data provision, analytics) on vertical axis and the spectrum from narrow to wide BI on horizontal axis. Include ETL, DWH, Reporting, OLAP, Data Mining, BSC

Why Do Companies Need BI?

Common Complaints

- Reports arrive **too late**
- Numbers are **inconsistent** across departments
- Data is **inaccurate** or incomplete
- Analysis is done manually in Excel — error-prone and slow
- Every department has its own *version of the truth*

The Finance Reporting Problem

Without a DWH, a controller spends up to **48 %** of their time on data collection and validation — instead of on actual analysis. (Hackett Group Study)

Goal of BI

Fast, consistent, fact-based information for everyone who needs to make decisions.

Information Supply Chain

Just as goods flow through a supply chain, **information flows** through the organisation.

Picture: Information Supply Chain

Add the Information Supply Chain diagram: Data → Analysis → Knowledge → Action, with “Operative Business Processes” underneath as the source. Show the cycle.

Roles in the Information Supply Chain

Role	Responsibility
Business User	Consumes business information
Reporter	Creates standard reports
Guide	Provides detailed information
Miner / Analyst	Data analysis and discovery
Administrator	Infrastructure and application management

Where Does the Data Come From? Operational Systems (OLTP)

Online Transaction Processing (OLTP)

The **source** of data for the DWH. OLTP systems run the day-to-day business: orders, payments, inventory, HR. They are optimised for **fast writes and concurrent users**, not for analytics.

Property	OLTP (Source)	DWH / OLAP (Target)
Purpose	Run the business	Analyse the business
Operations	INSERT/UPDATE/DELETE	Mainly SELECT
Data	Current, single record	Historical, aggregated
Volume	MB – GB	GB – TB
Users	Clerks, ERP apps	Analysts, managers
Response time	Milliseconds	Seconds – minutes
Examples	SAP, Salesforce, POS	DWH, OLAP cubes, BI tools

The Data Warehouse: Definition

Bill Inmon's Definition (1992)

A Data Warehouse is a **subject-oriented**, **integrated**, **time-variant**, and **non-volatile** collection of data in support of management's decision-making process.

In practice, a DWH is...

- A central **data pool** that forms the basis of information supply in the enterprise
- Sourced from operational systems (ERP like SAP) and external data (market data, competitor information)
- The **foundation** from which reports, analyses, and dashboards are built

The Four Characteristics of a DWH

1. Subject-Oriented

Data is organised around key business subjects — Customer, Product, Sales, Service — not around applications or processes. Operational systems are process-oriented; the DWH is subject-oriented.

2. Integrated

Data from heterogeneous sources is cleaned, transformed, and stored in a **consistent, unified format**. Names, units, and encodings are harmonised across all sources.

Picture: Subject-Oriented vs. Operational

Add a diagram showing: Operational apps (Sales, Finance, Marketing, Hotline, Training) on the left pointing into the DWH on the right, which is organised by subjects: Customer, Service, Product.

The Four Characteristics of a DWH (cont.)

3. Time-Variant

Every data record is associated with a **time stamp or validity period**. The DWH stores **multiple snapshots** of data over time — unlike operational systems, which typically reflect only the current state. Years or even decades of history are retained.

4. Non-Volatile

Data is **loaded** into the DWH and then **only read**. There are no updates or deletes in the transactional sense. The data is stable and persistent within a time unit. Loading happens in **defined intervals** (daily, weekly, ...).

Examples

Summary The DWH stores *detailed, filtered, subject-related data*, derived aggregates and reports, metadata, historical data (multiple years), and pre-aggregated data for fast querying.

Application Areas of the Data Warehouse

The most common use cases are in the business / management area.

Main Areas

- **Controlling:** Data basis for financial reporting, KPIs, budget vs. actuals, consolidation
- **Customer Relationship Management (CRM):** Customer segmentation, direct marketing, churn prediction, customer profitability analysis
- **Knowledge Management:** Combining structured data (DWH) with unstructured data (documents, email, images) via document management and intranets
- **Management Concepts:** Balanced Scorecard, Business Performance Management, Total Quality Management support

Typical Pain Points in Controlling without a DWH

- Heterogeneous data sources: ERP, SAP, Excel, local databases
- Validity of financial figures is questionable
- Manual consolidation of data from many systems
- High time expenditure for report preparation
- Up to **48 %** of controller time goes to data gathering, not analysis (Hackett Group)

Goal of Controlling in BI

The DWH automates data consolidation and provides a **single source of truth** so controllers can focus on actual analysis and decision support.

Use Case: Customer Relationship Management (CRM)

The CRM Data Challenge

Customer data is scattered across many systems:

- Transactional data in operational applications (orders, accounts)
- Offers in Word documents
- Revenue / product statistics in the DWH
- Contact notes as emails and private files
- Customer self-presentation on the web

What BI enables in CRM

- Customer segmentation and profiling
- Direct marketing campaign management
- Shopping basket / cross-sell analysis
- Customer profitability and credit scoring
- Churn (defection) prediction
- Service quality and response analysis

ROI and Cost Considerations

Return on Investment (ROI)

- A measure of the profitability of invested capital
- Only quantifiable aspects are considered
- DWH projects often have a difficult-to-quantify benefit side
- Expected ROI materialises **over time** — not immediately

Picture: ROI Curve over Time

Add a cost/benefit curve diagram: X-axis = time, Y-axis = cost/benefit. Show the initial investment dip, the break-even point, and the rising return-on-investment as the DWH matures.

Typical Cost Categories

- **Up-front:** Planning, modelling, design
- **One-time:** Hardware, software, data cleansing
- **Ongoing:** Data maintenance, data quality, DWH management, training, support

Table of Contents

- 1 Introduction
- 2 Business Intelligence & Data Warehouse
- 3 DWH Architectures**
- 4 Data Modelling
- 5 Building a Data Warehouse
- 6 OLAP & Business Intelligence

Approaches to Building a Decision-Support System

There is no single “correct” DWH architecture. The right choice depends on the organisation’s size, budget, maturity, and data strategy.

Four Main Approaches

- 1 **Virtual Data Warehouse** — direct access to operational systems
- 2 **Independent Data Marts** — separate marts per department
- 3 **Monolithic Enterprise DWH** — single centralised warehouse
- 4 **Integrated (Evolutionary) Architecture** — EDW + dependent data marts

Key Insight

The evolution of most DWH landscapes follows this sequence. Organisations often start with option 1 or 2 and eventually move towards option 4.

Architecture 1: Virtual Data Warehouse

Concept

Instead of physically copying data into a warehouse, queries are directed **directly to the operational source systems** — or to extracts thereof. No central data store is created.

Picture: Virtual DWH Architecture

Add a diagram showing operational source systems (Orders, Products, Customers, Accounts, Invoices) on the left, connected via direct query/access arrows to BI tools (Finance, Marketing, Sales) on the right. Label it “Virtual Warehouse”.

Advantages

- Low development effort and cost
- Fast results

Disadvantages

- Uncertain data quality
- Burdens operational systems
- **No historical data available**

A virtual DWH is not really a warehouse.

Architecture 2: Independent Data Marts

Concept

Each department builds its own data mart, sourced **directly from operational systems**. There is no central integration layer.

Picture: Independent Data Marts

Add a diagram showing operational source systems feeding directly into separate Finance, Marketing, and Sales data marts. Show many-to-many extraction lines forming a “spider web”.

The $m \times n$ Extraction Problem

With m independent data marts and n source systems, up to $m \times n$ extraction interfaces are needed. This quickly becomes an unmanageable web of connections.

Independent Data Marts: Pros and Cons

Advantages

- Lower costs
- Shorter implementation time
- Fast ROI per department

Disadvantages

- Fragmented, isolated data (“data silos”)
- Multiple conflicting “versions of the truth”
- Redundant and inefficient extracts
- No consistent quality standards
- Expensive and redundant storage
- Maintenance becomes a nightmare
- Long-term costs outweigh short-term benefits

Architecture 3: Monolithic Enterprise Data Warehouse

Concept

A single, large, centralised data warehouse that integrates all enterprise data. End users query the warehouse directly — there are no separate data marts.

Picture: Monolithic EDW

Add diagram: operational sources feeding into one large central Enterprise Data Warehouse, with BI tools querying it directly. No data marts layer.

Advantage

Integrated and consistent data across all departments.

Disadvantages

- Not suited for end-user queries (too complex/detailed)
- Conflicting requirements from different departments
- Poor query response times
- Long project duration before any business value

Architecture 4: Integrated (Hub-and-Spoke)

Concept

The recommended architecture. The **Enterprise Data Warehouse (EDW)** acts as the central hub, and **dependent data marts** are built on top of it for departmental use. Built **iteratively**: start with one data mart, expand step by step.

Picture: Hub-and-Spoke Architecture

Add diagram showing: Operational sources → EDW (Hub) → Finance Mart, Marketing Mart, Sales Mart, Personal Data Marts (Spokes). Show metadata flowing horizontally. Include ODS as an optional component between sources and the EDW.

The Integrated Architecture: Building Iteratively

How the Integrated Architecture Grows

- 1 Start with an **enterprise-wide subject model** and infrastructure plan
- 2 Build the **first data mart**, simultaneously creating the first layer of the EDW
- 3 Build the **second data mart**, extending the EDW with shared/common data
- 4 Continue adding marts and expanding the EDW incrementally
- 5 Eventually add an **ODS** for near-real-time operational reporting
- 6 Incorporate **external and unstructured data**

"Think Big – Start Small"

The iterative approach allows the organisation to learn, reduce risk, and deliver early value while building towards a comprehensive solution.

The Integrated Architecture: Pros and Cons

Advantages

- Consistent, integrated data
- Single source of truth
- Flexible and extensible
- Reduced up-front investment
- Early ROI via first data marts
- Lower risk via iterative delivery
- Data marts are optimised for end-user queries

Challenges

- Requires enterprise-wide vision and governance
- Higher initial planning effort
- Long path to full coverage
- Needs experienced architects

The Enterprise Data Warehouse (EDW)

Definition and Purpose

The EDW was introduced by Bill Inmon around 1990. It **centralises, standardises, and integrates** data from all operational sources.

Goals of the EDW

- Build the **history** that is missing in OLTP systems
- Integrate data from across the entire enterprise
- Provide a **clean and reliable foundation** for the warehouse environment
- Feed departmental and personal data marts
- Be as **flexible as possible** for future extensions

Important

The EDW is **not** intended for direct end-user queries. It is the backend engine that feeds the data marts.

The Data Mart

Definition

A **Data Mart** is a subset of the DWH environment focused on the data and analytical needs of a **specific department or business domain**.

Characteristics

- Smaller in scope than the EDW
- Aligned with a specific business requirement (Sales, Finance, HR)
- Minimises complexity of data access for end users
- Delivers good query performance
- Uses dimensional models (star/snowflake schema)

Types

Dependent	Fed from the EDW. Consistent with enterprise data.
Independent	Sourced directly from operational systems. Risk of data silos.
Personal	Individual analyst subset. Loosely governed.

The Operational Data Store (ODS)

Definition (Inmon, early 1990s)

The ODS is a **subject-oriented, integrated, volatile, and current** data store that supports operational decision making.

Key Properties

- Contains **current** (or very recent) integrated data — no full history
- Data is **updated frequently** (near real-time or hourly)
- Can **feed the DWH** with already-cleaned, integrated data
- Or can be fed *by* the DWH for operational reporting

ODS vs. EDW

ODS

Subject-oriented

Integrated

Temporary / volatile

Current data only

EDW

Subject-oriented

Integrated

Non-volatile

Full historical data

Metadata: The Data About Data

Definition

Metadata is descriptive information about the structure and content of data, and about the applications and processes that access it. (MDC OIM Specification)

Technical Metadata (maintained by IT)

- Storage location, infrastructure (network, servers)
- Number of columns and rows in a table
- Table structure, format, relationships
- ETL rules: extraction, transformation, load processes
- Creation date and update schedules

Business Metadata (maintained by users)

- Business rules, transformation logic, calculation descriptions
- Business-language descriptions of tables and fields
- Data ownership and security classifications
- Definition and purpose of each data element

The Reference Architecture

Picture: DWH Reference Architecture (Bauer/Günzel)

Add the “Referenzmodell für die DWH-Architektur” diagram showing: Data Sources → Extraction → Staging/Working Area → [Transformation] → Base Database → [Load] → Data Warehouse → [Load] → Analysis. Below: Monitor, DWH Manager, Metadata Manager, Repository. Use dashed lines for control flow and solid for data flow.

Layers in the Reference Architecture

- **Data Acquisition Layer:** Extraction + Staging + Transformation
- **Storage Layer:** Base Database + DWH
- **Analysis Layer:** Data Marts + OLAP + BI tools
- **Management Layer:** Monitor, DWH Manager, Metadata/Repository

Table of Contents

- 1 Introduction
- 2 Business Intelligence & Data Warehouse
- 3 DWH Architectures
- 4 Data Modelling**
- 5 Building a Data Warehouse
- 6 OLAP & Business Intelligence

Why Data Modelling Matters

Data modelling is the **first and most critical phase** in building a DWH. It determines what data is stored, how it is structured, and how easily users can analyse it.

The Logical Data Model...

- Is the blueprint of the DWH
- Is **requirements-driven**: what questions does the business need to answer?
- Defines which data and in which logical structure is stored in the DWH
- Strongly driven by user needs
- Is **critically important for project success**

Two Modelling Phases

- 1 **Logical modelling**: What data and how is it structured? (Subject model, dimensional model, ER model)
- 2 **Physical modelling**: How is it technically implemented? (Tables, aggregations, partitioning, storage type)

Step 1: Subject Modelling

What is a Subject?

A subject is an **important business area or theme** in the enterprise about which a lot of information is collected.

- **Persons:** Customer, Employee, Supplier
- **Things:** Product, Account, Contract
- **Processes:** Order, Payment, Delivery

The Subject Model...

- Documents the most important subjects and their relationships
- Covers the **whole enterprise** at a high level
- Is easy for business users to understand
- Is represented as a graphical diagram (5–15 subjects)

Examples

Tip In operational data models, subjects tend to appear as the **hub of multiple relationships** — many tables reference them. This is a good way to identify them.

Subjects Vary by Industry

Examples by Sector

Sector	Typical Subjects
Commercial firm	Customer, Product, Organisational unit, Sales
Public sector	Citizen, Department, Employee, Service
University	Student, Professor, Course, Subject area
Bank	Customer, Account, Advisor, Product, OU
Insurance	Customer, Policy, Claim, Agent, Premium

Important

Subjects must be identified **before** modelling begins. They require analysis of the business requirements and existing data models. Use interviews, workshops, and mind maps to elicit them.

Example Subject Model: Bank

Picture: Bank Subject Model

Add a subject model graph for a bank with nodes: Customer, Account, Product, Advisor, Branch. Show edges (relationships) between them. Label the edges (e.g. "has", "belongs to", "advises", "sells"). Keep it simple – one diagram fitting a slide.

Subject List Example (Bank)

Subject	Description	Related to
Customer	Person/org using bank services	Product, Account
Advisor	Bank employee serving customers	Branch, Customer
Branch	Organisational unit of the bank	Advisor, Customer
Account	Used to manage payment transfers	Customer, Product
Product	Services sold to customers	Customer, Account

Step 2: Dimensional Modelling

The Dimensional Data Model...

- Provides a **detailed logical view** of the data
- Divides data into **Facts** and **Dimensions**
- Is **requirements-oriented**: what analyses are needed?
- Is represented like an Entity-Relationship diagram
- The key difference from classical ER: the **degree of normalisation** is deliberately lower (denormalisation for query performance)

Core Concepts

Facts	The measurable business metrics to be analysed
Dimensions	The perspectives from which facts are analysed
Grain	The lowest level of detail stored (most important!)

Definition

Facts contain the **business metrics (KPIs)** to be analysed.

Sources and Types

- Either taken directly from operational systems: quantities, prices, measurements
- Or derived from them: revenue, profit margin, averages
- Stored at the **lowest level of detail** (finest grain) in the DWH

Additive Types of Facts

Type	Description	Example
Additive	SUM over all dimensions	Revenue, orders
Semi-additive	SUM over some dimensions only	Account balance
Non-additive	Cannot be meaningfully summed	Ratios, prices

Definition

Dimensions are the **logical perspectives** under which facts are analysed.

- Logical groupings of attributes: e.g. Year, Month, Day → **Time**
- Can contain additional descriptive attributes (e.g. customer first name)
- Each dimension gets its own table
- Dimensions often correspond to the **subjects** from the subject model
- Connected to the fact table via **dimension keys**

Examples

Typical Dimensions **Time** (Year/Quarter/Month/Day), **Product** (Category/Group/Individual Product), **Customer** (Segment/Region/Individual), **Geography** (Continent/Country/Region/City), **Organisation** (Company/Division/Department)

Dimension Hierarchies

Dimensions are organised in **hierarchies** — levels of detail from coarse to fine. Hierarchies are the basis for drill-down and roll-up.

Product Hierarchy

- Total
- Industry
- Product Group
- Product Family
- Product (lowest level)

Time Dimension: Multiple Hierarchies

- *Hierarchy 1:* Year → Quarter → Month → Day
- *Hierarchy 2:* Year → 4-4-5 Month → Week → Day

One dimension can contain **multiple** hierarchies.

Picture: Product Dimension Hierarchy

Add a tree/pyramid diagram for the Product dimension: Gesamt at top, then Branche, Produktgruppe, Produktfamilie, Produkt at bottom. Arrows pointing downward. Label it “Drill-Down” direction.

The Multidimensional Data Model

The dimensional model conceptually organises data as a **multidimensional space** (hypercube).

Picture: 3D Multidimensional Cube

Add a 3D cube diagram with three axes: Dimension 1 = Organisation (Branch/Region/Advisor), Dimension 2 = Product Line (Group/Individual), Dimension 3 = Time (Year/Month/Day). Highlight one cell with a value, e.g. "Org=Branch02, Product=03, Time=1999".

Key Concepts

- Each **cell** in the cube holds an aggregated measure
- **Sparse cubes**: many cells have no data (normal in practice)
- **Dense cubes**: most cells contain values
- Real cubes can have many more than 3 dimensions

The Star Schema

Structure

- One central **fact table** connected to multiple **dimension tables** via foreign keys
- Dimension tables are **denormalised** (flat, wide tables)
- Simple to understand and query
- Optimised for read performance
- Named for its star-like visual shape

Examples

Example: Wine Sales Fact table: FACT_SALES with columns Product-Key, Region-Key, Taste-Key, Time-Key, Revenue.

Dimension tables: DIM_PRODUCT (Rosé/White/Sparkling), DIM_REGION (Brandenburg/Hessen/...), DIM_TASTE (dry/semi-dry/mild), DIM_TIME (Jan/Feb/Mar/...)

Star Schema: Diagram

Picture: Star Schema Diagram (Wine Sales)

*Add the wine sales star schema diagram: Central fact table **FACT_SALES** with foreign keys to four dimension tables. **DIM_PRODUCT** on the left (Rotwein, Weißwein, Sekt), **DIM_REGION** on the right (Brandenburg, Hessen, Niedersachsen), **DIM_GESCHMACK** below-left (trocken, halbtrocken, mild), **DIM_ZEITRAUM** below-right (Januar, Februar, März). Connect all four to the central fact table.*

Fact Table Columns

- **Dimension keys** (FK): link to each dimension table
- **Degenerate dimensions**: attributes with no dimension table (e.g. invoice number)
- **Measures**: the actual facts (revenue, quantity, ...)
- **Derived values**: calculated metrics

The Snowflake Schema

Concept

The snowflake schema is a **normalised extension** of the star schema. Large or redundant dimension tables are split into multiple related tables.

Examples

Example: Region Dimension In a star schema, one flat DIM_REGION table contains (City, PostalCode, FederalState, Country).

In a snowflake schema, this splits into: DIM_REGION → DIM_POSTAL_AREA → DIM_CITY

Picture: Snowflake Schema Diagram

Add the snowflake extension of the wine sales star schema: DIM_REGION now has a FK to DIM_POSTAL_AREA, which has a FK to DIM_CITY. DIM_ZEITRAUM splits into Month-ID and Week-ID sub-tables. Show the “snowflake” branching effect.

Star vs. Snowflake Schema

Property	Star Schema	Snowflake Schema
Normalisation	Denormalised	Partially normalised
Query simplicity	Simple (fewer joins)	More complex (more joins)
Query performance	Faster	Slightly slower
Storage redundancy	Higher	Lower
Maintenance	Easier	More complex
Typical use	Smaller dimensions	Very large dimensions

In practice...

The star schema is the **most common choice** for data marts. The snowflake schema is used selectively to reduce storage for specific very large dimension tables. **Most real-world DWH schemas are hybrid.**

The Date Dimension

The time/date dimension is present in **almost every DWH**. It is pre-populated for all dates in the analysis range (e.g. 2000–2040).

Examples

Typical Date Dimension Table

Date	Weekday	Month	Cal.Q	Fin.Q	Week	4-4-5	Holiday
25.12.1997	Thu	12	4	1	52	4	Yes
26.12.1997	Fri	12	4	1	52	4	Yes
31.12.1997	Wed	12	4	1	53	4	No
01.01.1998	Thu	01	1	1	01	1	Yes

Important Note

Time is **not** an isolated dimension. Every other dimension can also change over time (available products, customer lists, sales territories). This is why **Slowly Changing Dimensions** are crucial.

Slowly Changing Dimensions (SCD)

The Problem

Dimension data changes over time. A customer moves cities. A product is re-categorised. A sales region is restructured. How do we handle this in a non-volatile DWH?

SCD Type 1 — Overwrite

Simply **update** the existing row. No history is retained.

Use when historical accuracy for this attribute does not matter.

Example: Fixing a data entry error.

SCD Type 2 — Add a New Row (most common)

Insert a **new row** for the new value. Old rows remain unchanged with a validity period (start/end date or a `is_current` flag).

Use when you need to track the history of changes.

Example: Customer changes address.

SCD Type 3 — Add a New Column

Dimension Keys: Natural vs. Surrogate Keys

Natural Key

The business identifier from the source system (e.g. customer number, product code, employee ID).

Surrogate Key

A **meaningless integer** generated by the DWH (e.g. 1, 2, 3, ...). It is the primary key of the dimension table inside the DWH.

Why Use Surrogate Keys?

- Natural keys can change in source systems — surrogate keys are stable
- Essential for **SCD Type 2**: multiple rows for the same business entity get different surrogate keys
- Smaller (integer) = faster joins
- Decouples the DWH from source system changes
- Allows integration of data from **multiple source systems** (different natural keys for the same entity)

Entity-Relationship Modelling in the DWH Context

When ER is Used

ER modelling (Peter Chen, 1976) is the traditional approach for relational data modelling. In the DWH context, it is used:

- For modelling the **integration/staging layer** (3NF)
- Favoured by the Inmon approach
- Particularly popular with database and IT specialists

Core ER Concepts

Entity	A distinguishable object (Customer, Product)
Attribute	A property of an entity (Name, Price)
Relationship	A connection between two entities
Cardinality	How many instances participate: 1:1, 1:n, n:m
Optionality	Whether participation is mandatory or optional

Normalisation in the DWH

Why Normalise?

- Avoids redundancy and inconsistency
- Prevents update/insert/delete anomalies
- Results in a clearer, more maintainable data model
- Concept introduced by E.F. Codd (1972)

Normal Forms Recap

Form	Rule
1NF	All attribute values are atomic (no repeating groups)
2NF	1NF + every non-key attribute fully depends on the whole PK
3NF	2NF + no non-key attribute depends on another non-key attribute
BCNF	Stronger form of 3NF

DWH vs. OLTP

The core DWH/staging layer uses **3NF** for clean data storage. Data marts use **deliberately denormalised** star/snowflake schemas for analytical query

Table of Contents

- 1 Introduction
- 2 Business Intelligence & Data Warehouse
- 3 DWH Architectures
- 4 Data Modelling
- 5 Building a Data Warehouse**
- 6 OLAP & Business Intelligence

How a Data Warehouse is Built

The Process: Iterative by Nature

Building a DWH is **never a one-time project** but an ongoing, iterative effort. New requirements, changed source systems, and evolving business needs continuously shape the DWH.

The 7 Phases (based on the Rapid Warehousing Methodology)

- 1 **Qualification:** Is the organisation ready for a DWH project?
- 2 **Requirements Analysis:** What data and analyses are needed?
- 3 **Design:** Architecture, data models, process design
- 4 **Implementation:** Build ETL processes, load data, develop marts
- 5 **Acceptance Test:** Validate quality and completeness
- 6 **Deployment:** Train users, go live
- 7 **Review:** Lessons learned, plan next iteration

Requirements Analysis: What Do We Need?

Goal

Understand **what questions the business needs to answer** and what data is required to answer them.

Key Activities

- Interview stakeholders: managers, analysts, power users
- Identify **KPIs** and reporting requirements
- Determine required **grain** (day, week, individual transaction)
- Define the **history depth** needed (1 year? 10 years?)
- Understand the required **refresh frequency**: daily batch, near-real-time, or on-demand

Examples

Good Questions to Ask Stakeholders

- *“What decisions does your department make every week?”*
- *“What data do you currently look up manually?”*

Source System Analysis

Goal

Understand the **data landscape**: what exists, where it lives, how good it is.

DWH Source Systems Can Be Very Diverse

- Relational databases: Oracle, DB2, SQL Server, PostgreSQL
- ERP / enterprise systems: SAP R/3, SAP BW, Siebel
- Legacy systems: IMS, VSAM, COBOL files
- Web / Internet sources: APIs, web scraping
- Flat files: Excel, CSV, Access, Lotus
- Unstructured text files

Common Surprise

Data quality in source systems is **almost always worse** than anyone expects.
Budget significant effort for data cleansing.

The ETL Process

ETL = Extract, Transform, Load

ETL is the central **data pipeline** that moves data from source systems into the DWH.

Picture: ETL Flow with Staging Area

Add a three-layer flow diagram: (Bottom) Operational source systems $\xrightarrow{E}$ Staging Area (raw landing zone) $\xrightarrow{T}$ Transformation (cleanse, harmonise, enrich) $\xrightarrow{L}$ Data Warehouse (storage + data marts). Use arrows labeled E, T, L. Add the “DWH Manager / Monitor” below as a control element.

Key Principle: Decouple Extraction from Transformation

Extracting data into the staging area first, before transforming it, allows **validation, profiling, and error handling** without impacting either the source system or the target DWH.

Extract: Getting the Data Out

The First Step

Extraction creates access to the operational data sources and copies data into the staging area.

Extraction Methods

Method	Description
Full Refresh (Data Refresh)	Load all data from scratch on each run
Delta / Incremental Update	Load only changes since the last run

Change Data Capture (CDC)

- **Timestamp columns:** filter rows where `UPDATED_AT > last_run`
- **Database triggers:** source system records changes automatically
- **Transaction log mining:** read the DB log to find changed rows
- **Full diff:** compare snapshots (last resort, expensive)

Extract: Light Transformations During Extraction

Some minor transformations can already be applied during extraction. These are technical in nature and do not touch business logic.

Acceptable During Extraction

- Character encoding conversion: ASCII to/from EBCDIC
- Decompression
- Case conversion (usually to upper case)
- Date and time format conversion
- Sorting

Warning

Applying too much transformation during extraction can:

- **Obscure errors** — bad data should be visible in staging
- **Impact performance** of the operational source system

Keep extraction *as light as possible*.

Data Quality: The Biggest Challenge

Why Data Quality Matters

- **40 %** of companies have suffered losses due to poor data quality (The Data Warehousing Institute)
- **15–20 %** of values in typical customer databases are incorrect (Uni Trier)
- **50 %** of CRM projects fail because of data quality problems (Wayne Eckerson, TDWI)

Common Data Quality Issues

Issue	Description
Completeness	Missing or NULL values
Consistency	Same entity represented differently across sources
Accuracy	Values that are factually wrong or outdated
Timeliness	Data that arrives too late to be useful
Duplicates	Same record represented multiple times

Data Defect Classification

Three Classes of Defects

	Class 1	Class 2	Class 3
Detection	Automatic	Automatic	Manual
Correction	Automatic	Manual	Manual
Syntactic	Known format errors	Detectable format incompatibilities	—
Semantic	Missing values (fill)	Outliers / implausible values	Undetected semantic errors in source

Class 3 Defects

Class 3 defects (undetected semantic errors) **must be fixed at the source system** — they cannot be reliably corrected in the ETL pipeline. This requires cooperation with the source system owners.

Data Wrangling: What is it?

Definition

Data Wrangling (also called data munging) is the process of transforming and mapping raw data into a format suitable for analysis. In the DWH context, this is the heart of the **Transform** step in ETL.

Typical Wrangling Activities

- **Filtering:** Remove irrelevant or erroneous rows
- **Standardisation:** Unify naming conventions and formats
- **Harmonisation:** Align data from multiple sources into one consistent schema
- **Enrichment:** Add derived or calculated attributes
- **Aggregation:** Pre-compute summary values
- **Deduplication:** Identify and remove duplicate records

Data Standardisation: Why it Matters

The Problem

The same company can appear in data under many different names:

Examples

Revenue Report Without Standardisation

Company Name	Revenue (€)	Invoice #
BMW	15,687	123-65
Bayrische Motorenwerke	47,657	875-5
BMW	134,982	123-410
b.m.w.	9,807	459-60
Bayerische Motoren Werke	237,897	164-45
WHERE Company = 'BMW'	150,669	(only 2 rows!)
After standardisation	446,030	(all 5 rows)

Filtering in the Staging Area

After extraction, validation routines are run to detect and correct defects automatically where possible. Both syntactic and semantic defects can be found.

Harmonisation

The process of **business-level reconciliation** of filtered data. After technical standardisation, values from different sources are aligned to the same business meaning:

- “M” and “male” and “männlich” → M
- “DE”, “Germany”, “Deutschland”, “DEU” → DE
- Date formats: 15/03/2024, 2024-03-15, 20240315 → ISO 8601:
2024-03-15

Deduplication: Matching Data Records

The Problem

The same real-world entity appears multiple times in the data — with slight variations in spelling, format, or completeness.

Duplicate Matching Steps

- 1 Define **match criteria**: which fields to compare? (name, address, date of birth, ...)
- 2 Generate a **match key (match code)** per record based on the criteria
- 3 Compare records with identical or similar match codes
- 4 Assign a **confidence score** based on field weights
- 5 Records above the threshold are flagged as duplicates
- 6 Choose the **master record**; merge or delete duplicates

Impact

Duplicate customer records lead to: incorrect campaign targeting, inflated customer counts, wasted marketing spend.

Load: Bringing Data into the DWH

What the Load Step Does

After transformation, cleaned and enriched data is loaded into the DWH target tables.

Loading Dimension Tables

- Look up existing dimension records by natural key
- If found: apply SCD logic (Type 1 = update, Type 2 = insert new row)
- If not found: insert new record, generate new surrogate key

Loading Fact Tables

- Resolve all dimension keys (look up surrogate keys)
- Append new fact rows (facts are typically never updated)
- Handle late-arriving data: facts that arrive after their dimension records have been updated

Loading OLAP Cubes

ETL vs. ELT

Traditional ETL

Transform data **before** loading into the DWH.

Transformation runs in a separate ETL server or tool.

Only clean data enters the DWH.

Typical in on-premise environments.

Modern ELT (Extract, Load, Transform)

Load raw data **first**, then transform it *inside* the DWH using its compute power.

Raw data is preserved for re-processing. Enabled by powerful cloud DWH engines (Snowflake, BigQuery).

	ETL	ELT
Transform where?	External tool	Inside DWH engine
Data in DWH	Clean only	Raw + clean
Flexibility	Lower	Higher
Typical context	On-premise	Cloud
Typical tools	Informatica, SSIS	dbt, Spark, BigQuery

Data Quality as a Continuous Process

Data quality is **not a one-time fix** — it requires an ongoing improvement cycle.

Picture: Data Quality Cycle

Add a circular diagram with three phases: (1) Analysis (profile and measure quality), (2) Model Building (define rules and correction logic), (3) Automation (implement rules in ETL pipeline). Use circular arrows to show the iterative loop.

Strategies for Sustained Data Quality

- Validate at load time; reject or quarantine bad records
- Maintain an **error/quarantine table** for rejected rows
- Implement **data quality dashboards** and metrics
- Work with source system owners to fix issues *at the source*
- Document **data lineage**: where does each column come from?

DWH Processes Must Be Managed

The ETL pipeline consists of many interdependent processes. Managing them requires:

- Definition of **execution order** and timing
- Support for **parallel processing**
- Handling of **dependencies** between jobs
- Definition of **error handling** and alternative paths

The Role of Metadata in ETL

Metadata (especially technical metadata) is the control layer:

- Every process records its **input and output tables**
- ETL jobs can be **driven by metadata** (rules, mappings stored centrally)
- Impact analysis: “if I change column X, which jobs are affected?”
- Change management is much easier with fine-grained, documented processes

Aggregations

What is an Aggregation?

An aggregation is the **summation or consolidation** of data along dimension hierarchies. For example: summing daily sales to monthly, then annual totals.

Why Pre-Compute Aggregations?

- Aggregating billions of rows at query time is **very slow**
- Pre-computed aggregations can be served from much smaller tables
- The tradeoff: **storage** for **speed**
- The level of aggregation has a **huge impact** on data volume

Examples

Simple Aggregation Example

Customer	Product	Sales (units)
Meier	Refrigerator	5
Meier	Microwave	7
Müller	Oven	6

Table of Contents

- 1 Introduction
- 2 Business Intelligence & Data Warehouse
- 3 DWH Architectures
- 4 Data Modelling
- 5 Building a Data Warehouse
- 6 OLAP & Business Intelligence**

What is OLAP?

Online Analytical Processing (OLAP)

OLAP is a technology (hardware and software) for:

- **Simple and fast** data analysis
- From **arbitrary perspectives** and at different levels of detail
- Without having to define perspectives and levels in advance

⇒ **Dynamic analysis of multidimensional data!** (based on Codd's 12 OLAP rules, 1993)

The Foundation of OLAP

The basis of OLAP is always a **dimensional data model** — a star or snowflake schema (or a multidimensional database structure built from it).

Typical OLAP Questions

- *What were total sales per region per quarter in 2024?*
- *Which product category grew the fastest over the last 3 years?*
- *Compare revenue in Germany vs. France for Q2 this year.*

Codd's 12 OLAP Rules

Edgar F. Codd (inventor of the relational model) defined 12 rules for a true OLAP system in 1993:

- ➊ Multidimensional view of data
- ➋ Transparent data access
- ➌ Unrestricted data availability
- ➍ Consistent performance
- ➎ Client/server architecture
- ➏ Support for default dimensions (e.g. Time)
- ➐ Dynamic sparsity handling
- ➑ Multi-user support
- ➒ Unrestricted operations across all dimensions
- ➓ Intuitive data manipulation
- ➑ Flexible reporting
- ➒ Unlimited dimensions and aggregation levels

The OLAP Cube

What is an OLAP Cube?

An OLAP cube is a **specialised storage format** with the following properties:

- Matrix-like format optimised for multidimensional access
- Enables rapid extraction of pre-summarised data at multiple levels
- Organises data in **hierarchies**
- Structures data according to **dimensions and measures**

Picture: 3D OLAP Cube with Axes

Add a 3D cube diagram with three labelled axes: Axis 1 = Geography (Country/Region/City), Axis 2 = Product Line (Category/Group/Product), Axis 3 = Time (Year/Quarter/Month). Highlight one cell and label it with an example value: "Geography=Germany, Product=Electronics, Time=2024 Q1: Revenue = €1.2M".

OLAP Cube: From Table to Cube

The same data can be represented as a **relational table** or as a **multidimensional cube** — they contain the same information.

Picture: Cube-Table Duality

Add the “Dualismus Würfel-Tabelle” diagram: Left side: a 3D cube with axes *Product*, *Time*, *Geography*. Right side: a flat relational table with columns *Article*, *Branch*, *Date*, *Sales*. Show that the cells in the cube correspond to rows in the table. Example rows: *Radio/Heidelberg/3.1.2003/7*, *Radio/Mannheim/3.1.2003/4*.

Tabular Representation

OLAP results can be displayed as:

- Simple 2D cross-tables (Rows = Products, Columns = Regions)
- Nested tables (multiple dimensions on each axis)

Term	Meaning
Dimension Name	The label of a dimension (e.g. “Time”)
Level	A level of detail within a dimension (e.g. Year, Quarter, Month)
Hierarchy	The arrangement of levels within a dimension
Member	A single category value within a level (e.g. “Q1 2024”)
Measure	The numeric value being analysed (e.g. Revenue, Quantity)
Crossing	A specific combination of dimension levels (defines a cell)
N-Way Crossing	The combination of all lowest levels — the most granular data
Sparsity	The proportion of cells in the cube that have no value
Drill-Through	Access to the underlying transaction-level detail data

Picture: Time Dimension with Levels and Hierarchies

Add a dimension tree for the Time dimension: Top: Dimension name "YQ Zeit". Below: two hierarchies side by side. Hierarchy 1 (YQ): 2003/2004/2005 → Q3/Q4/Q1/Q2 (Quarters). Hierarchy 2 (YM): 2003/2004/2005 → Jan/Feb/Mar/Apr/May/Jun/Jul/Aug... Label: "Member" for individual values (e.g. Q3, Jan), "Level" for the row type, "Hierarchy" for the overall branch.

Key Insight

A single dimension can contain **multiple hierarchies**. Example for Time:

- Hierarchy 1 (Calendar): Year → Quarter → Month → Day
- Hierarchy 2 (Fiscal): Fiscal Year → 4-4-5 Month → Week → Day

OLAP Operation: Roll-Up and Drill-Down

Roll-Up (Aggregation)

Move **up** in the dimension hierarchy.
Less detail, more aggregation.

Example:

Day → Month → Quarter → Year

Drill-Down (Disaggregation)

Move **down** in the dimension hierarchy.
More detail, less aggregation.

Example:

Year → Quarter → Month → Day

Picture: Roll-Up / Drill-Down on Product Hierarchy

Add a vertical arrow diagram on the Product hierarchy: (Top) Total → Industry → Product Group → Product Family → (Bottom) Product. Left-side arrow pointing up: "Roll-Up". Right-side arrow pointing down: "Drill-Down".

OLAP Operations: Slice, Dice, and Pivot

Slice — Fix One Dimension

Select a **single value** on one dimension, reducing the cube by one axis.

Example: “Show me only Q1 2024” (fixes the Time axis).

Result: a 2D table from a 3D cube.

Dice — Filter Multiple Dimensions

Apply **multiple filters** across several dimensions, producing a sub-cube.

Example: “Q1 and Q2, Electronics and Clothing, Germany only.”

Result: a smaller cube with restricted ranges.

Pivot (Rotate) — Swap Axes

Re-orient the result by swapping row and column dimensions.

Example: Switch from Rows=Products/Columns=Regions to Rows=Regions/Columns=Products.

No data changes — only the presentation perspective changes.

OLAP Storage Variants: Overview

Three Technical Approaches

How the multidimensional data is **physically stored** determines performance, scalability, and flexibility.

Variant	Storage Type
MOLAP	Multidimensional (proprietary cube database)
ROLAP	Relational (star/snowflake schema in RDBMS)
HOLAP	Hybrid (combination of both)

Picture: OLAP Scalability Matrix

Add the scalability diagram: X-axis = Size of data / tables, Y-axis = Query complexity. MDDDB OLAP (MOLAP) in bottom-left (small data, complex queries), Hybrid OLAP (HOLAP) in middle, Relational OLAP (ROLAP) in top-right (large data, simpler queries). Use overlapping circles or regions.

MOLAP: Multidimensional OLAP

How it Works

Data is stored in a **proprietary multidimensional database** (cube file). The cube is pre-built and pre-aggregated. No traditional table structure — dimensions and values are stored together. All aggregation levels may be pre-computed in one cube.

Advantages

- Optimised for access via dimensions
- Very fast for complex queries
- Good for many dimensions
- Good for deep hierarchies
- Good for sparse data (low cardinality)

Disadvantages

- Poor scalability
- Problematic for dimensions with very many values (high cardinality)
- Size and performance issues at scale

ROLAP: Relational OLAP

How it Works

Data is stored in a standard **relational database (RDBMS)**. The star schema is used (or fully denormalised tables). OLAP cube operations are translated into SQL queries at runtime. Views are used to represent different perspectives.

Advantages

- Excellent scalability
- Works well for very large datasets (high cardinality)
- Flat hierarchies are handled well
- Highly dynamic (views can change without rebuilding)
- Good for simple queries

Disadvantages

- Slower for very complex queries
- Not ideal for many dimensions simultaneously
- Deep hierarchies cause many joins
- Sparse data is inefficient

HOLAP: Hybrid OLAP

The Best of Both Worlds

HOLAP combines **multidimensional** and **relational** storage. The idea is to use each where it performs best:

- **MOLAP** storage for highly aggregated data and complex queries
- **ROLAP** storage for detailed transaction-level data and large datasets

When to Use HOLAP

- When you need **both** summary speed and detail scalability
- Complex analytical queries on summaries → use MOLAP layer
- Mass queries on millions of detail rows → use ROLAP layer

Examples

Practical Assessment HOLAP is **pragmatic and effective** for most enterprise DWH scenarios. The choice of storage type should be driven by the actual query patterns and data characteristics — not by vendor preference.

SQL Extensions for OLAP: ROLLUP

Standard SQL has powerful OLAP extensions built into most modern databases.

GROUP BY ROLLUP

Generates subtotals and a grand total along a dimension hierarchy in a single query.

```
SELECT region, product, SUM(revenue) AS total_revenue
FROM fact_sales
GROUP BY ROLLUP(region, product);
```

Result Structure

- One row per (region, product) combination
- One subtotal row per region (product = NULL)
- One grand total row (region = NULL, product = NULL)

This replaces multiple UNION ALL subqueries with a single, optimised pass over the data.

SQL Extensions for OLAP: CUBE and GROUPING SETS

GROUP BY CUBE

Generates **all possible combinations** of grouping attributes.

```
SELECT region, product, year, SUM(revenue)
FROM fact_sales
GROUP BY CUBE(region, product, year);
```

GROUPING SETS

Specify exactly **which grouping combinations** to compute. More flexible than ROLLUP or CUBE.

```
SELECT region, product, year, SUM(revenue)
FROM fact_sales
GROUP BY GROUPING SETS (
    (region, year),      -- by region and year
    (product, year),     -- by product and year
    (region),            -- by region only
    ()                   -- grand total
);
```

SQL Window Functions

What are Window Functions?

Window functions perform calculations **across a set of related rows** without collapsing them into a single output row. They use the `OVER()` clause.

```
-- Running total of revenue per region
SELECT
    date_key, region, revenue,
    SUM(revenue) OVER (
        PARTITION BY region
        ORDER BY date_key
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total,
    LAG(revenue) OVER (
        PARTITION BY region ORDER BY date_key
    ) AS prev_period_revenue
FROM fact_sales;
```

Common Window Functions

`ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`, `LAG()`, `LEAD()`, `NTILE()`,

From Data to Decision: The BI Layer

What the BI Layer Does

The BI layer sits **on top of** the DWH and OLAP infrastructure. It presents data to business users in a consumable format. Target users are non-technical: managers, analysts, executives.

Picture: Complete BI Stack

Add a vertical stack diagram from bottom to top: (1) Source Systems (OLTP, SAP, CRM) → (2) ETL / ELT Pipeline → (3) Data Warehouse (Staging, Core, Data Marts) → (4) OLAP / Semantic Layer (Cube or SQL views) → (5) BI Tools (Dashboards, Reports, Ad-hoc queries) → (6) Business Decision. Add example product names at each layer (e.g. Airflow, Snowflake, Power BI).

Types of BI Analysis

Descriptive Analytics — *What happened?*

Summarises historical data. Standard reports and dashboards.

Example: Monthly revenue report.

Diagnostic Analytics — *Why did it happen?*

Drill-down analysis to identify root causes.

Example: Revenue dropped in Q3 — which region? Which product?

Predictive Analytics — *What will happen?*

Statistical models and ML to forecast future outcomes.

Example: Demand forecasting, churn prediction.

Prescriptive Analytics — *What should we do?*

Recommends actions based on data and models.

Example: Automated pricing engine, inventory optimisation.

Types of BI Reports and Dashboards

Standard / Operational Reporting

- Fixed-format, scheduled reports delivered regularly
- Example: End-of-day sales summary, monthly P&L
- Designed once, run repeatedly

Ad-Hoc Analysis

- Users create their own queries and reports on demand
- Requires a user-friendly **semantic layer**
- No IT involvement needed for each analysis

Interactive Dashboards

- Visual, real-time display of KPIs
- Users can filter, drill down, and explore data
- Tools: **Power BI, Tableau, Looker, Metabase, SAP BusinessObjects**

KPIs and Metrics

Key Performance Indicator (KPI)

A quantifiable measure used to evaluate success in achieving a business objective. Not all metrics are KPIs — only those that are directly tied to strategic goals.

Properties of a Good KPI

- **Specific:** tied to a concrete business goal
- **Measurable:** expressed as a number
- **Actionable:** something the business can actually influence
- **Timely:** available when decisions need to be made
- **Consistently defined:** same meaning across all departments

Examples

Example KPIs Monthly Recurring Revenue (MRR), Customer Acquisition Cost (CAC), Net Promoter Score (NPS), Inventory Turnover Rate, Customer Lifetime Value (CLV), Sales Conversion Rate.

The Semantic Layer

Definition

The **semantic layer** (business layer) maps the physical DWH tables and columns to **business-friendly terms** that non-technical users can understand and trust.

What the Semantic Layer Provides

- Abstracts SQL complexity from business users
- Defines **metrics, KPIs, and dimensions** in business language
- Ensures consistent definitions company-wide: “Revenue” always means the same thing everywhere
- Enables self-service analytics without IT involvement

Examples

Example Products **SSAS Tabular** (Microsoft), **Looker LookML**, **dbt metrics layer**, **Power BI datasets**, **SAP BW BEx** (Business Explorer)

Summary: From Raw Data to Business Decision

Picture: End-to-End Flow Diagram

Add a horizontal arrow diagram showing the complete journey: (1) Raw Transactions (OLTP/ERP) → (2) ETL/ELT with Data Quality → (3) Data Warehouse (Staging + Core + Data Mart) → (4) OLAP Cube / Semantic Layer → (5) BI Dashboard / Report → (6) Business Decision. Add small icons or example artefacts at each step.

Key Takeaways

- **BI** is the “intelligence service” of the enterprise
- The **DWH** is the foundation: integrated, historical, non-volatile
- **ETL and data quality** determine the value of all analytics
- **OLAP** enables fast, flexible, multidimensional analysis
- **BI tools** translate data into decisions for business users